

SKB_Project_S33843

Pi-hole DNS Ad Blocker on NixOS

Author: Tufan Yeşilhüyük **Repository:** <https://github.com/turpitufo/skb-project>

1. Problem Description

Modern networks suffer from pervasive DNS-based tracking, advertisement delivery, and telemetry exfiltration. Most of this traffic occurs silently in the background on every connected device — phones, laptops, smart TVs — without the user's awareness. Browser-level ad blockers address only a fraction of this, leaving apps, firmware, and background processes completely unfiltered.

This project implements a network-wide DNS sinkhole using Pi-hole on NixOS, intercepting DNS queries from all devices on the local network and blocking known ad, tracking, and malware domains before any connection is established. The solution operates at the network layer, requiring no configuration on individual client devices.

2. Use Cases and System Analysis system

roles:

- **Pi-hole server** — DNS resolver, DHCP server, blocklist manager
- **Network clients** — any device on the LAN (laptop, phone, IoT)
- **Administrator** — manages blocklists, reviews query logs, configures

allowlists via web UI

cases:

UC1 — Network-wide ad blocking A client device sends a DNS query for

`doubleclick.net`. Pi-hole intercepts it, matches it against the blocklist, and returns `0.0.0.0`. The connection is never established. The client receives no ad.

UC2 — Legitimate DNS resolution A client queries

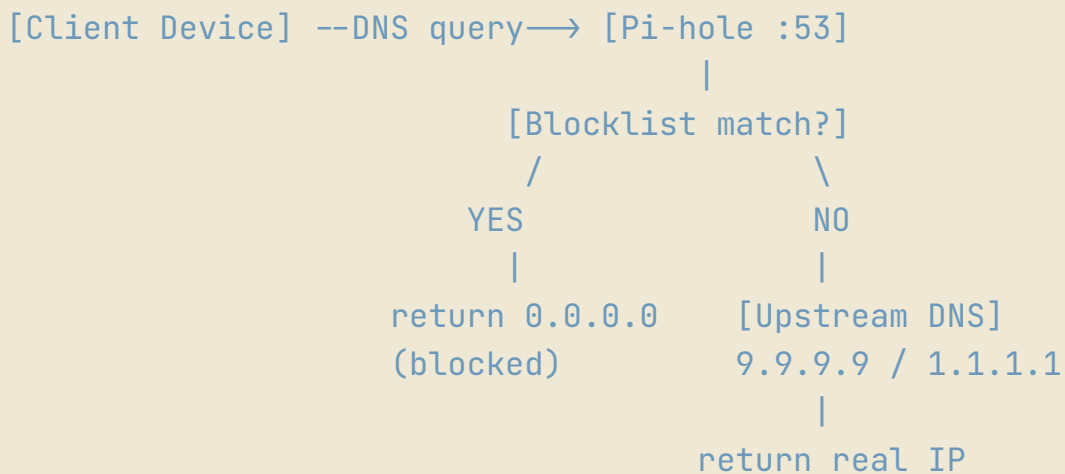
`google.com`. Pi-hole finds no match in the blocklist and forwards the query to the upstream resolver

(9.9.9.9). The real IP is returned and the connection proceeds normally.

UC3 — DHCP lease assignment A new device joins the network. Pi-hole's DHCP server assigns it an IP address from the configured range and hands out 192.168.0.171 as its DNS server. The device immediately routes all DNS through Pi-hole with no manual configuration.

UC4 — Administrator monitoring The administrator opens https://192.168.0.171 and reviews the live query log, blocked domain statistics, and per-device traffic breakdown.

UC5 — Blocklist management The administrator adds or removes blocklist URLs via the web UI or declaratively via configuration.nix. Pi-hole updates its gravity database and begins enforcing the new rules immediately.



3. System Architecture

Protocols used:

- DNS (UDP/TCP port 53) — core query interception and resolution
- DHCP (UDP port 67) — network address and DNS assignment
- HTTPS (TCP port 443) — encrypted web admin interface
- SSH (TCP port 22) — remote headless administration

Technologies:

- NixOS 25.05 — declarative Linux, entire system defined in configuration.nix
- Pi-hole FTL 6.6.2 — DNS resolver and DHCP server

- NixOS firewall (nftables) — port filtering and access control
- `systemd` — service management and autostart

Security mechanisms:

- DNS-level packet filtering — blocks millions of known malicious and tracking domains
- Firewall rules — only required ports exposed, all others closed by default
- Password hashing — web UI password stored as `BALLOON-SHA256` hash, never plaintext
- SSH key-only authentication — password login disabled on the server
- Read-only config enforcement — `misc.readOnly = true` prevents runtime tampering with Pi-hole's configuration

Blocklists:

- StevenBlack unified hosts
 - Hagezi Pro (ads, tracking, telemetry)
 - Hagezi Threat Intelligence (malware, phishing)
 - OISD Big (comprehensive)
 - Hagezi Social (social media trackers)
-

4. Description of Implementation

This project was originally planned for a Radxa Zero (512MB, 4×1.8GHz, aarch64) — a Raspberry Pi Zero equivalent. The board was not delivered in time, but the spirit of the project remains intact. Thanks to the reproducible nature of NixOS, deploying to the Radxa Zero when it arrives requires copying the same `configuration.nix` and running `nixos-install`. Nothing else changes.

NixOS is a declarative Linux distribution using the Nix language to configure the entire system — packages, services, kernel, firewall — in a single file. This was designed for reproducibility, atomic upgrades, and rollbacks.

There were four machines involved:

1. **pihole** — NixOS virtual machine (development and testing)
2. **Hal** — host laptop, final deployment target (NixOS)
3. **pNix** — client laptop (NixOS)

4. Samsung Galaxy Note9 — mobile client

Although the virtual machine was used for development, WiFi bridging proved impossible at the driver level — the VM received a NAT address (`192.168.122.x`) invisible to the real LAN. The config was then applied directly to the host machine, which was already on the real LAN at `192.168.0.171` . Since both machines run NixOS, this was a single file copy and rebuild.

The minimal NixOS ISO requires no additional packages. The Pi-hole service is fully declarative:

```
services.pihole-ftl = {  
  enable = true;  
  settings = {  
    dns.upstreams = [ "9.9.9.9" "1.1.1.1" ];  
    misc.readOnly = true;  
  };  
};
```

After `nixos-install` , the service was active on first boot with no further configuration.

Notable NixOS-specific challenges:

Setting the web UI password was unexpectedly difficult. Standard documentation suggests running `pihole-FTL --config webserver.api.password` , but NixOS sets `misc.readOnly = true` by default, making the config immutable at runtime. The binary is also not on PATH — it lives deep in the Nix store at `/nix/store/<hash>/bin/pihole-FTL` . The solution was to temporarily disable `readOnly` , generate the password hash via the store binary, paste the resulting `BALLOON-SHA256` hash into `configuration.nix` as `webserver.api.pwhash` , then re-enable `readOnly` and rebuild.

Another NixOS-specific issue: `systemd-resolved` holds port 5353 by default, and `libvirt` 's internal `dnsmasq` was bound to `192.168.122.1:53` , both preventing Pi-hole from binding to port 53. The resolved stub listener was disabled via `DNSStubListener=no` , and the libvirt default network was destroyed:

```
sudo virsh net-destroy default
sudo virsh net-autostart default --disable
```

NetworkManager also overrides `/etc/resolv.conf` with the router's DHCP-provided DNS, ignoring `networking.nameservers`. The fix was `networking.networkmanager.insertNameservers`, which injects the desired server at the top of the resolver list regardless of DHCP.

The ISP-provided router (Compal, PLAY Poland) does not expose DNS configuration in its DHCP settings and does not allow DHCP to be disabled. Pi-hole's built-in DHCP server was enabled and configured to serve the entire `192.168.0.50-250` range. The router's DHCPv4 is now disabled and fully replaced by Pi-hole, which hands out `192.168.0.171` as DNS to every device on the network automatically.

One remaining limitation: the router's DHCPv6 is set to stateless (SLAAC), meaning devices may receive IPv6 addresses and potentially route DNS queries over IPv6, bypassing Pi-hole entirely. This is known as an IPv6 leak. It will be resolved when the Radxa Zero arrives and connects via ethernet, enabling proper IPv6 DNS configuration.

DNS resolution was verified using `dig`; returns `0.0.0.0` if blocked, returns real IP if resolved:

```
→ dig @192.168.0.171 doubleclick.net

; <<>> DiG 9.20.22 <<>> @192.168.0.171 doubleclick.net
; (1 server found)
;; global options: +cmd
;; Got answer:
;; —>HEADER<— opcode: QUERY, status: NOERROR, id: 36218
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0,
ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;doubleclick.net.                IN      A

;; ANSWER SECTION:
```

```

doubleclick.net.          2      IN      A      0.0.0.0

;; Query time: 403 msec
;; SERVER: 192.168.0.171#53(192.168.0.171) (UDP)
;; WHEN: Mon Jun 08 08:10:44 CEST 2026
;; MSG SIZE rcvd: 60

~
→ dig @192.168.0.171 google.com

; <<>> DiG 9.20.22 <<>> @192.168.0.171 google.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; —>HEADER<— opcode: QUERY, status: NOERROR, id: 31485
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL:
1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                 153     IN      A      142.251.98.100
google.com.                 153     IN      A      142.251.98.113
google.com.                 153     IN      A      142.251.98.138
google.com.                 153     IN      A      142.251.98.139
google.com.                 153     IN      A      142.251.98.101
google.com.                 153     IN      A      142.251.98.102

;; Query time: 412 msec
;; SERVER: 192.168.0.171#53(192.168.0.171) (UDP)
;; WHEN: Mon Jun 08 08:10:52 CEST 2026
;; MSG SIZE rcvd: 135

```

5. AI Tools Used

Tool: chat.mistral.ai

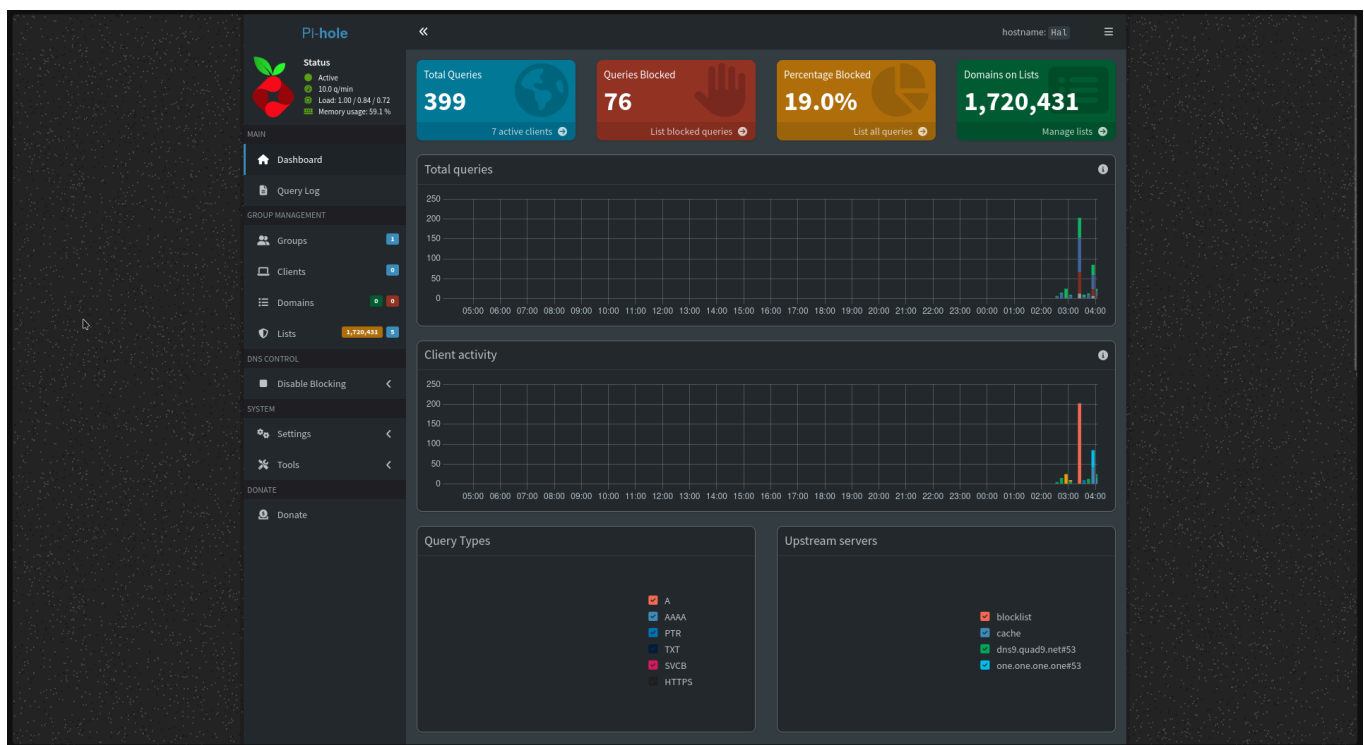
Purpose: Used throughout the entire implementation as a real-time debugging and configuration assistant.

Specifically used for:

- Configuring NixOS for DHCP
- Investigating the connection issues
- Rewriting this document in the required format and in a more professional language

6. Screenshots

6.1 Working solution



```

~ took 9s
x sudo ping pagead2.googlesyndication.com
PING pagead2.googlesyndication.com (0.0.0.0): 56 data bytes
64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.041 ms
64 bytes from 127.0.0.1: seq=1 ttl=64 time=0.049 ms
64 bytes from 127.0.0.1: seq=2 ttl=64 time=0.047 ms
64 bytes from 127.0.0.1: seq=3 ttl=64 time=0.040 ms
64 bytes from 127.0.0.1: seq=4 ttl=64 time=0.042 ms
^C
--- pagead2.googlesyndication.com ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 0.040/0.043/0.049 ms

```

The screenshot shows a web browser displaying the NixOS Wiki page for 'pihole-ftl'. The page includes sections for 'Contents', 'Beginning', 'Minimal Configuration Example', and 'Adding lists and enabling web interface'. A terminal window is overlaid on the page, showing the following commands and output:

```

d: nu — Konsole
IP4.DNS[1]:
IP6.DNS[1]:
IP6.DNS[2]:
0

~
~ nmcli dev show | grep DNS
IP4.DNS[1]:
IP6.DNS[1]:
IP6.DNS[2]:
0

~

```

The terminal output shows the DNS configuration for the network interface 'eth0'. The output is as follows:

```

IP4.DNS[1]: 192.168.0.1
IP6.DNS[1]: 2a02:a302:0:1::10
IP6.DNS[2]: 2a02:a302:0:1::10
0

```

The terminal also shows the output of the command 'nmcli dev show | grep DNS', which returns the following information:

```

~ nmcli dev show | grep DNS
IP4.DNS[1]: 192.168.0.171
IP6.DNS[1]: 2a02:a302:0:1::10
IP6.DNS[2]: 2a02:a302:0:1::10
0

```

The terminal window also shows the output of the command 'nmcli dev show | grep DNS', which returns the following information:

```

~ nmcli dev show | grep DNS
IP4.DNS[1]: 192.168.0.171
IP6.DNS[1]: 2a02:a302:0:1::10
IP6.DNS[2]: 2a02:a302:0:1::10
0

```



```
05:12 [Di] [🔊] [📶] [28]

.      512462  IN      NS      c.root-s
ervers.net.
.      512462  IN      NS      d.root-s
ervers.net.
.      512462  IN      NS      e.root-s
ervers.net.
.      512462  IN      NS      f.root-s
ervers.net.
.      512462  IN      NS      g.root-s
ervers.net.
.      512462  IN      NS      h.root-s
ervers.net.
.      512462  IN      NS      i.root-s
ervers.net.
.      512462  IN      NS      j.root-s
ervers.net.
.      512462  IN      NS      k.root-s
ervers.net.
.      512462  IN      NS      l.root-s
ervers.net.
.      512462  IN      NS      m.root-s
ervers.net.

;; Query time: 336 msec
;; SERVER: 192.168.0.171#53(192.168.0.171) (UDP)
;; WHEN: Mon Jun 08 05:10:48 CEST 2026
;; MSG SIZE rcvd: 239

- $ dig @192.168.0.171 doubleclick.net

; <<>> DiG 9.20.23 <<>> @192.168.0.171 doubleclick.net
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 4564
8
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0
, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;doubleclick.net.          IN      A

;; ANSWER SECTION:
doubleclick.net.          2       IN      A      0.0.0.0

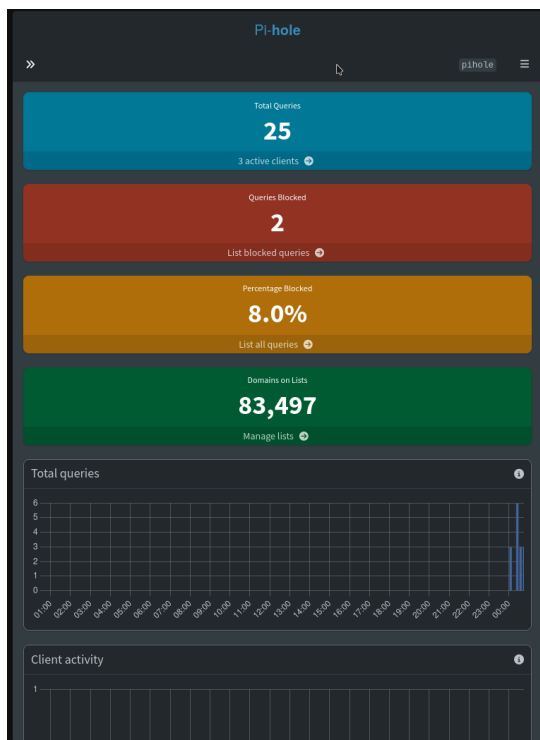
;; Query time: 180 msec
;; SERVER: 192.168.0.171#53(192.168.0.171) (UDP)
;; WHEN: Mon Jun 08 05:11:06 CEST 2026
;; MSG SIZE rcvd: 60

- $ █

ESC      /      -      HOME      ↑      END      PGUP

⌘      CTRL  ALT      ←      ↓      →      PGDN
```

6.2 The process



```
Hal : hu — Konsole
Hal on ~ main [!] took 39s
➜ dig @192.168.122.59 google.com

;<> Di6 9.20.23 <> @192.168.122.59 google.com
; (1 server found)
; global options: +cmd
; Got answer:
; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 46656
; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: udp: 1232
; XDR: 3 (Status Answer)
;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                0      IN      A      142.251.98.113
google.com.                0      IN      A      142.251.98.139
google.com.                0      IN      A      142.251.98.138
google.com.                0      IN      A      142.251.98.102
google.com.                0      IN      A      142.251.98.101
google.com.                0      IN      A      142.251.98.100

;; Query time: 2 msec
;; SERVER: 192.168.122.59#53(192.168.122.59) (UDP)
;; WHEN: Mon Jun 08 01:00:40 CEST 2026
;; MSG SIZE rcvd: 141

Hal on ~ main [!]
➜ dig @192.168.122.59 doubleclick.net

;<> Di6 9.20.23 <> @192.168.122.59 doubleclick.net
; (1 server found)
; global options: +cmd
; Got answer:
; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 14860
; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: udp: 1232
;; QUESTION SECTION:
;doubleclick.net.          IN      A

;; ANSWER SECTION:
doubleclick.net.          2      IN      A      0.0.0.0

;; Query time: 0 msec
;; SERVER: 192.168.122.59#53(192.168.122.59) (UDP)
;; WHEN: Mon Jun 08 01:00:50 CEST 2026
;; MSG SIZE rcvd: 60
```



Pi-hole

Log in (uses cookie)

[Documentation](#) [GitHub](#) [Pi-hole Discourse](#)

Donate if you found this useful.

```
(admin) 192.168.122.59 — Konsole

fig webserver.api.pwhash
$BALL00N-SHA256$v=1$s=1024,t=32$user0IgVx1fKkJUeI80ledcA==$xQ2Ycm3UZwHILrNzAehN/jbIzrReaib5JnFnI8Emn6o=

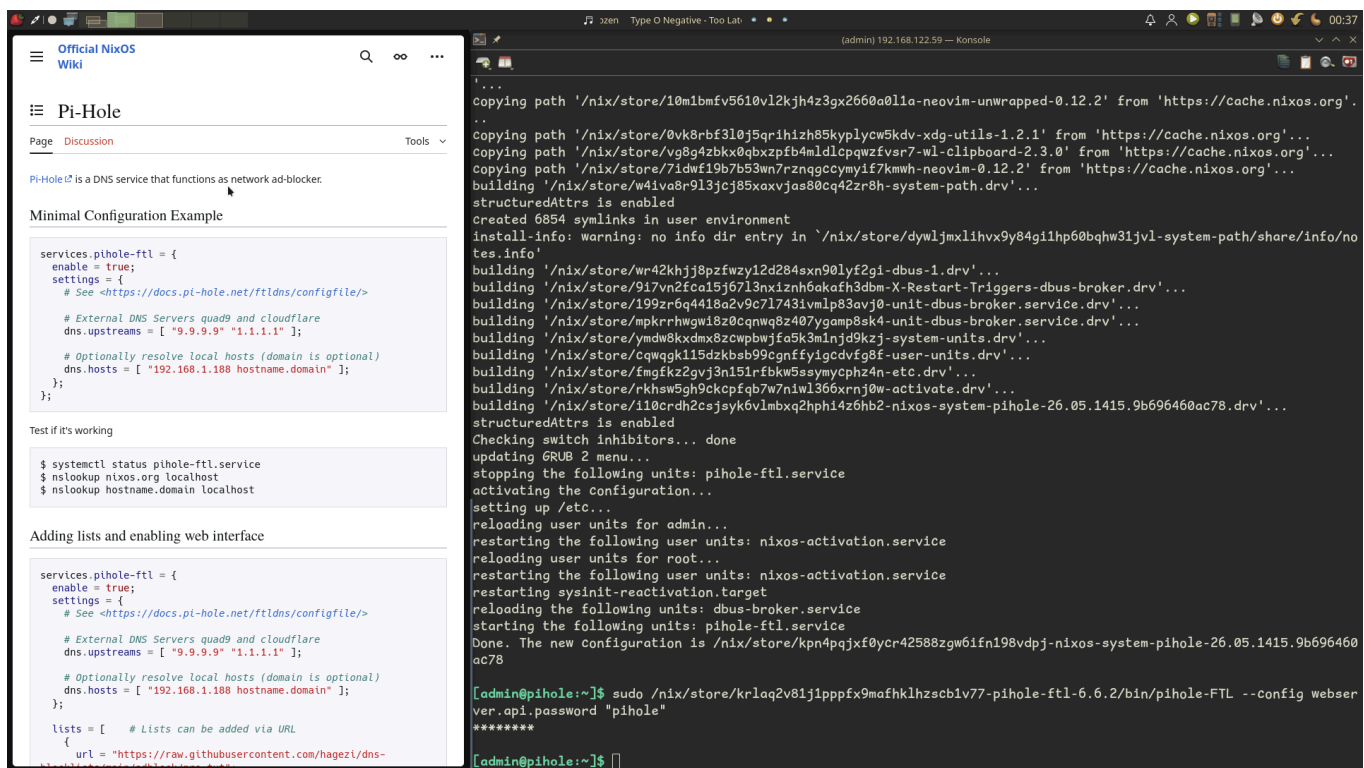
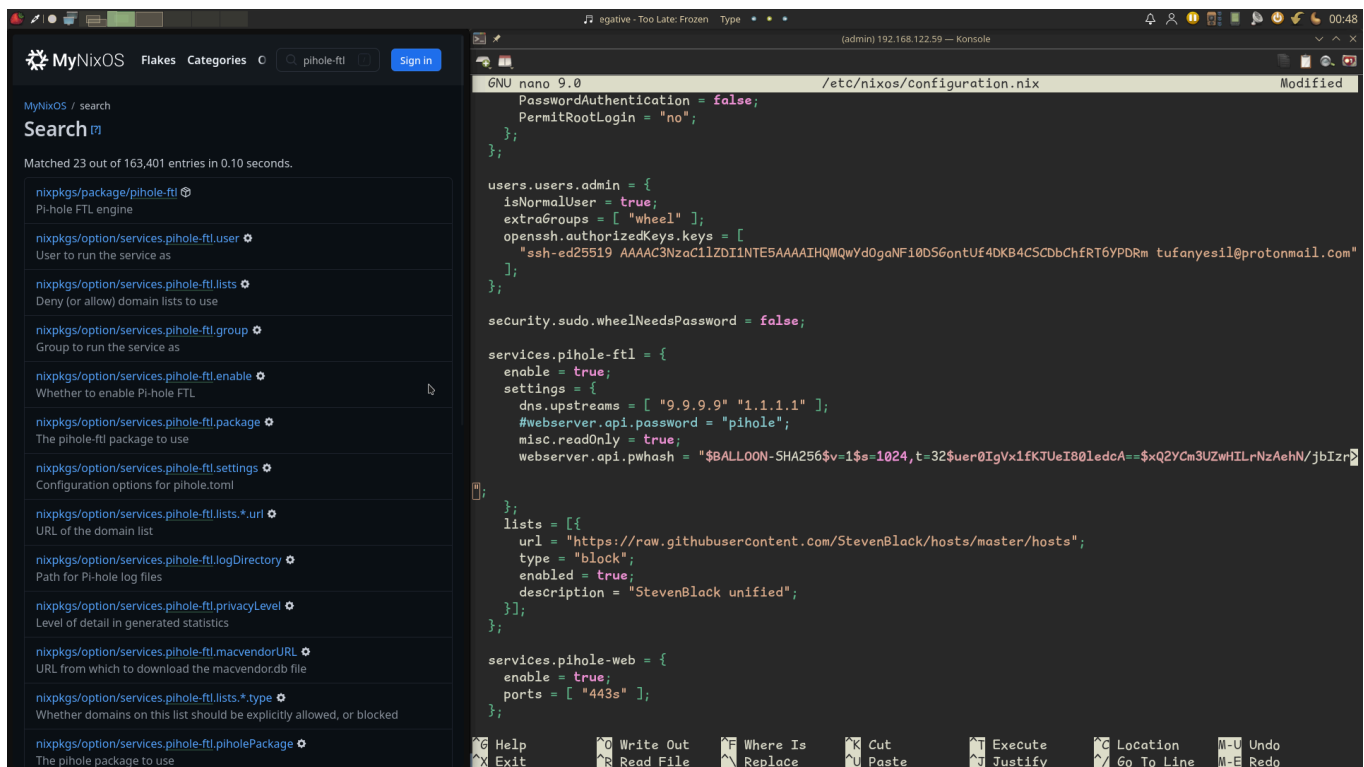
[admin@pihole:~]$ sudo nano /etc/nixos/configuration.nix

[admin@pihole:~]$ sudo nixos-rebuild switch
building the system configuration...
these 6 derivations will be built:
/nix/store/9g5ippzxa63y83kc541vb8jw110xszejx-pihole.toml.drv
/nix/store/39mfs9zl7qv119wla3w89mqgvzy621xn-unit-pihole-ftl.service.drv
/nix/store/k03hvpq71qd90zdxqqj8id96nlbydf3q-system-units.drv
/nix/store/a8n7znmfrag4jlcqll1422vf8bkkbp18-etc.drv
/nix/store/9pvs561r0s419hjr18iipiggn8p3b0-activate.drv
/nix/store/xzq30zv6ybvdly220f8s501np59fbx9-nixos-system-pihole-26.05.1415.9b696460ac78.drv
building '/nix/store/9g5ippzxa63y83kc541vb8jw110xszejx-pihole.toml.drv'...
structuredAttrs is enabled

building '/nix/store/39mfs9zl7qv119wla3w89mqgvzy621xn-unit-pihole-ftl.service.drv'...
building '/nix/store/k03hvpq71qd90zdxqqj8id96nlbydf3q-system-units.drv'...
building '/nix/store/a8n7znmfrag4jlcqll1422vf8bkkbp18-etc.drv'...
building '/nix/store/9pvs561r0s419hjr18iipiggn8p3b0-activate.drv'...
building '/nix/store/xzq30zv6ybvdly220f8s501np59fbx9-nixos-system-pihole-26.05.1415.9b696460ac78.drv'...
.
structuredAttrs is enabled
Checking switch inhibitors... done
updating GRUB 2 menu...
stopping the following units: pihole-ftl.service
activating the configuration...
setting up /etc...
reloading user units for admin...
restarting the following user units: nixos-activation.service
reloading user units for root...
restarting the following user units: nixos-activation.service
restarting sysinit-reactivation.target
starting the following units: pihole-ftl.service
Done. The new configuration is /nix/store/7rk80sdpyfqm5sic21wf3k0h96yxcvvh-nixos-system-pihole-26.05.1415.9b696460ac78

[admin@pihole:~]$ sudo nixos-rebuild switch^C

[admin@pihole:~]$
```



7. Conclusions and Future Improvements

The project successfully implements network-level DNS filtering and DHCP management using Pi-hole on Minimal NixOS ISO, covering DNS protocol communication, HTTPS for the web interface, SSH for headless management, packet filtering, traffic blocking, firewall rules, and a full client-server

architecture — satisfying requirements 1, 2, 3a and 3b of the project specification.

The path was not straightforward. What started as a planned deployment on embedded ARM hardware became a multi-machine debugging session involving a virtual machine, two NixOS laptops, an ISP-locked router, and a series of NixOS-specific challenges that standard Pi-hole documentation does not cover. Every problem had a declarative solution — which is both the strength and the learning curve of NixOS.

Future improvements:

- Move Pi-hole to the Radxa Zero connected via ethernet, eliminating the IPv6 leak and the dependency on the host laptop being powered on
- Enable DNS-over-TLS so Android's Private DNS feature can point to Pi-hole system-wide without per-network manual configuration
- Add Unbound as a local recursive resolver, removing reliance on upstream providers like Quad9 and Cloudflare entirely
- Set up WireGuard so Pi-hole protects all devices even outside the home network